Evidence-Based Claim Verification System

System Overview

Purpose

This system verifies factual claims against provided evidence using multiple machine learning models and rule-based heuristics.

It classifies claims into three categories:

- **SUPPORTS**: Evidence confirms the claim
- **REFUTES**: Evidence contradicts the claim
- **NOT ENOUGH INFO**: Evidence is insufficient for verification

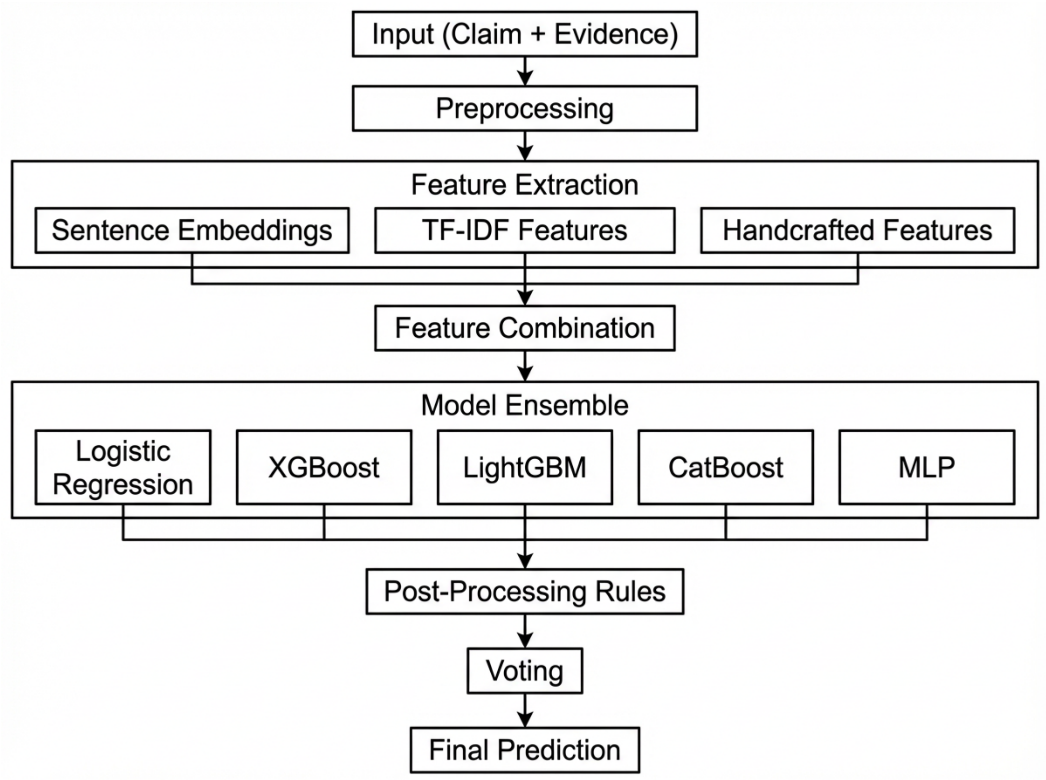
Key Features

- Multi-model ensemble approach (5 models)
- Semantic similarity analysis using transformer embeddings
- TF-IDF based linguistic features
- Rule-based post-processing for edge cases
- Interactive Streamlit web interface

Technology Stack

- **Core**: Python 3.x
- **ML Frameworks**: Scikit-learn, XGBoost, LightGBM, CatBoost, PyTorch
- **NLP**: SentenceTransformers, TfidfVectorizer
- **Web Interface**: Streamlit
- **Data Processing**: Pandas, NumPy, SciPy

System Architecture



Components

1. **Data Layer**

   - Raw dataset (train_data.csv)
   - Cached embeddings (.npy)
   - Saved models (.joblib, .pth)
   - TF-IDF vocabulary

2. **Processing Layer**

   - Text cleaning
   - Handcrafted features
   - TF-IDF vectorization
   - SBERT semantic embeddings
   - Sparse/dense feature integration

3. **Model Layer**

   - Logistic Regression
   - XGBoost
   - LightGBM
   - CatBoost
   - Deep MLP (PyTorch)

4. **Inference Layer**

    - Ensemble prediction
    - Contradiction-override logic
    - Final decision module

5. **UI Layer**

    - Streamlit frontend

Data Pipeline & Preprocessing

Sampling & Shuffling

- Maximum sample size: **250,000**
- Deterministic shuffle (seed=42)
- Stratified resampling for balancing REFUTES

Text Cleaning

Operations performed:

- HTML tag removal
- URL removal
- Special character stripping
- Whitespace compression
- Lowercasing

Label Encoding

- Converts class labels to numeric (0-N)

Verifiability Mapping

Rule-based:

- “verifiable” → 1
- “non-verifiable”, “not verifiable” → 0

Semantic Embeddings (SBERT)

Model used:

all-MiniLM-L6-v2

Features extracted:

- Claim embedding
- Evidence embedding

TF-IDF Features

- 5000 features
- N-grams (1 to 3)
- English stopwords removed
- Stored via .joblib + .npz

Handcrafted Features

Feature	Meaning
verifiable_flag	From user/verifiable input
num_contradiction	Checks disjointness of numbers across claim/evidence
semantic_conflict	SBERT cosine similarity thresholds
negation_flag	Mismatch of negation words
directnegation_flag	Direct "not X" contradictions

The handcrafted component provides embedding-based semantics.

Feature Fusion

Model combines 4 categories:

1. **Interaction features:** claim_emb * evidence_emb
2. **Raw embeddings:** claim_emb, evidence_emb
3. **Cosine similarity + handcrafted flags**
4. **TF-IDF features**

Model Parameters

Model Type	Key Hyperparameters	Training Strategy
Logistic Regression	max_iter = 1000; class_weight = balanced	Single-shot training
XGBoost	n_estimators = 800; learning_rate = 0.03; max_depth = 4	Early stopping
LightGBM	n_estimators = 800; learning_rate = 0.03; num_leaves = 31	Multi-logloss evaluation
CatBoost	iterations = 900; learning_rate = 0.02; depth = 10	Auto class weights; best-model selection
MLP (Neural Net)	Layers: 256 → 128 → 64 → output; dropout = 0.25–0.35	Early stopping (patience = 8)

Logistic Regression

Acts as a **baseline semantic–lexical classifier** that quickly captures linear relationships between combined embeddings, TF-IDF features, and handcrafted contradiction signals.

Advantages

- **Fast and lightweight** → ideal for real-time inference.
- **Interpretable** → easy to understand which features affect decision.
- Handles **high-dimensional sparse TF-IDF** well.
- Provides **stable baseline predictions** for ensemble voting.

Trade-offs

- Cannot model **non-linear interactions** between claim & evidence.
- Limited ability to capture **deep semantic contradictions** beyond surface patterns.
- Performance ceiling is lower than tree-based or neural models.

XGBoost

Functions as a **non-linear decision engine** that exploits complex feature interactions across SBERT embeddings, handcrafted features, and TF-IDF.

Advantages

- Excellent at **capturing interaction patterns** (e.g., negation + low similarity).
- Robust to noise in handcrafted features.
- Strong generalization on structured + text-derived features.
- Often gives **sharp REFUTES/SUPPORTS boundaries**.

Trade-offs

- Training is heavier than LR or LightGBM.
- Tends to overfit if feature space is very large (your stack is 6.5k+ dims).
- Less interpretable compared to LR or CatBoost.

LightGBM

Provides a **fast, memory-efficient booster** optimized for large, sparse TF-IDF matrices combined with dense embeddings.

Advantages

- Very fast training and inference → ideal for scaling to 250k+ samples.
- Handles **sparse TF-IDF** exceptionally well.

- Typically more memory-efficient than XGBoost.
- Performs well on **mixed dense + sparse input**, which matches your feature fusion.

Trade-offs

- More sensitive to **hyperparameter choices** than XGBoost.
- Can produce unstable decision boundaries on small classes.
- Slightly weaker than CatBoost in class-imbalanced scenarios.

CatBoost

Serves as a **stability-focused gradient booster** that handles imbalances & noise gracefully, especially helpful for REFUTES/NEI boundary cases.

Advantages

- Best handling of **imbalanced labels** (auto class weights).
- Smooth, stable performance across noisy handcrafted features.
- Less prone to overfitting compared to XGBoost/LightGBM.
- Great Out-of-Box performance with minimal tuning.

Trade-offs

- Training time can be higher than LightGBM.
- Slightly slower inference.
- Feature importances are interpretable but still not as transparent as LR.

MLP

It learns **non-linear semantic relationships** that tree models may miss. Acts as the **semantic–interaction learner** that fuses:

- SBERT embeddings
- interaction features
- contradictions signals
- TF-IDF

Advantages

- Captures **deep semantic patterns** between claim & evidence.
- Learns smooth representations across the fused feature space.
- Works well with **dense embedding-driven** inputs.
- Early stopping + dropout helps generalization.

Trade-offs

- More sensitive to feature scaling & embedding distributions.
- Requires more compute (slower to train).

- Less interpretable than LR or tree-based models.
- Performance can degrade if handcrafted signals dominate embeddings.

Final Ensemble

Each model independently predicts

Then a **post-processing override** layer acts like a vigilant referee:

Override rules

1. **Strong negative semantic conflict**
   → REFUTES
2. **Negation mismatch**
   → REFUTES
3. **Direct negation conflict**
   → REFUTES
4. **Numeric contradiction**
   → REFUTES

Only if none of these rules fire then model outputs remain untouched. Whichever label appears most frequently among:

- LR
- XGBoost
- LightGBM
- CatBoost
- MLP

becomes the **Final Prediction.**